GUÍA DE INTEGRACIÓN PARA DESARROLLADORES

API OCR de Pasaportes

Extracción de la página de datos de pasaportes de cualquier país con PaddleOCR, Qwen3-VL y validación MRZ (open source)

ENDPOINT PRINCIPAL	POST http://172.10.30.17:18474/pasaporte
AUTENTICACIÓN	Header X-API-Key
FORMATO DE RESPUESTA	JSON
VERSIÓN DE LA API	0.2.0
VERSIÓN DEL DOCUMENTO	1.0 · 27 de octubre de 2026
ELABORADO POR	Alexis Eduardo Juárez Jaimes

Uso interno. Contiene la especificación técnica del servicio; no contiene datos de clientes.

Contenido

1. Qué hace esta API . . . 3
1.1 Cómo funciona por dentro . . . 3
1.2 Documentos soportados . . . 3
2. Inicio rápido . . . 3
3. Conexión . . . 4
3.1 Endpoints . . . 4
3.2 Límites y tiempos . . . 4
4. Petición . . . 5
4.1 Ejemplos por lenguaje . . . 5
5. Estructura de la respuesta . . . 6
5.1 Bloque datos . . . 7
5.2 Bloque sello . . . 8
5.3 Bloque mrz . . . 8
5.4 Bloque validacion . . . 9
5.5 avisos, clasificacion, confianza y meta . . . 9
6. Códigos de respuesta . . . 9
6.1 Documento procesado (trae datos) . . . 10
6.2 Documento evaluado pero no extraído (sin datos) . . . 10
6.3 Errores (sin datos, con su estado HTTP) . . . 11
7. Integración recomendada . . . 11
7.1 Cómo decidir . . . 11
7.2 Reintentos . . . 12
7.3 Concurrencia . . . 12
7.4 Buenas prácticas . . . 12
8. Ejemplos de respuesta . . . 12
8.1 Código 200: procesado . . . 13
8.2 Código 210: con discrepancias . . . 14
8.3 Código 211: con avisos . . . 15
8.4 Código 212: incompleto . . . 16
8.5 Código 225 y 221: sin datos . . . 17
8.6 Errores . . . 17
9. Health check . . . 17
10. Seguridad . . . 18
11. Precisión y limitaciones . . . 18
12. Preguntas frecuentes . . . 19
13. Prompt para implementar con IA . . . 19
14. Glosario . . . 22

1. Qué hace esta API

La API recibe un expediente escaneado en PDF y devuelve en JSON los datos de la **página de datos del pasaporte**, de cualquier país. Si el expediente trae otros documentos (tarjeta de residente del INM, credencial INE, solicitudes, recibos, etc.), esas páginas se ignoran y no se extrae nada de ellas.

Todo corre en el servidor de la empresa con modelos **open source** (PaddleOCR y Qwen3-VL): los documentos no salen a ningún servicio externo y no hay costo por página.

1.1 Cómo funciona por dentro

Paso	Qué ocurre
1. Recepción	Valida la API key, que el archivo sea PDF, su tamaño y que se pueda abrir.
2. Localización	PaddleOCR lee las páginas y ubica la zona MRZ del pasaporte (2 líneas «P<...» al pie de la página de datos). Si la hoja viene girada prueba 90°, 180° y 270°; si aun así no la encuentra, el modelo de visión ubica el pasaporte en la página.
3. Lectura	El modelo de visión Qwen3-VL lee los campos impresos de la página de datos.
4. Validación MRZ	Lee la MRZ y verifica sus cinco dígitos de control. Corrige errores típicos de lectura (O/0, I/1, S/5...) solo cuando el dígito de control lo confirma.
5. Sello de cotejo	Si el PDF trae el sello «COTEJADO CONTRA ORIGINAL» del banco, lee la fecha, quién revisó y su clave.
6. Comprobaciones	Cruza la hoja de datos contra la MRZ y la CURP (fecha, sexo, número personal) y marca coincidencias, discrepancias y lo que no se pudo verificar.
7. Respuesta	Entrega el JSON con un código que indica la calidad de la extracción.

1.2 Documentos soportados

- **Pasaportes de cualquier país** con zona MRZ de 2 líneas (formato ICAO TD3). Probado con más de 40 países: México, Colombia, Venezuela, Argentina, España, Cuba, Corea, Chile, Perú, Brasil, Estados Unidos...
- **Pasaportes mexicanos:** además se extrae y valida la CURP.
- **No soportados:** tarjeta de residente del INM, credencial INE, licencias y cualquier otro documento: devuelven el código **225**.

> **Importante**
> La página de datos debe incluir las **2 líneas MRZ del pie** y venir en las primeras 4 páginas del PDF. La MRZ es la base de la validación: sin ella los datos se entregan, pero sin las comprobaciones contra la MRZ (códigos 212 o 210).

2. Inicio rápido

- Solicita la **API key** al responsable del servicio. Guárdala en una variable de entorno o gestor de secretos, nunca en el código.
- Envía el PDF por **POST multipart/form-data** en el campo `file` a `/pasaporte` con el header `X-API-Key`.
- Lee `codigo` para decidir (sección 6) y toma los datos de `datos`.

Ejemplo (curl)

```
curl -X POST "http://172.10.30.17:18474/pasaporte" \
  -H "X-API-Key: $OCR_PASAPORTE_API_KEY" \
  -F "file=@expediente.pdf;type=application/pdf"
```

3. Conexión

Concepto	Valor
URL base	`http://172.10.30.17:18474`
Red	Red interna de la empresa (servidor srviatwo). El servicio no está expuesto a internet.
Protocolo	HTTP (sin TLS). Ver sección 10, Seguridad.
Autenticación	Header `X-API-Key` con la clave del servicio. Sin ella se responde 401.
Formato de respuesta	JSON UTF-8, siempre con el mismo sobre (sección 5).
Documentación interactiva	Swagger en `http://172.10.30.17:18474/docs` y especificación OpenAPI en `http://172.10.30.17:18474/openapi.json`

3.1 Endpoints

Método	Ruta	Auth	Uso
POST	`/pasaporte`	Sí	**Endpoint principal.** Usar este en todas las integraciones.
POST	`/extract`	Sí	Alias para pruebas. Responde exactamente igual que /pasaporte.
GET	`/health`	No	Estado del servicio, para monitoreo.

3.2 Límites y tiempos

Concepto	Valor	Qué pasa si se excede
Tamaño máximo	25 MB por archivo	Código 413
Tipos aceptados	Solo PDF	Código 400 (imágenes) o 222 (archivo que no es PDF)
Archivos por petición	1	Solo se procesa el campo file
Páginas	Hasta 30 por archivo. Se analizan las primeras 4: el pasaporte debe venir ahí	Código 224
Tiempo de respuesta	10 a 25 segundos por expediente (típico 15 a 20)	-
Peticiones simultáneas	Hasta 2 se procesan a la vez	Las demás esperan su turno (ver 7.3)
Timeout recomendado en el cliente	120 segundos	-

4. Petición

Elemento	Valor
Método y ruta	`POST /pasaporte`
Content-Type	`multipart/form-data`
Header obligatorio	`X-API-Key: <tu clave>`
Campo del archivo	`file` (obligatorio). Si falta o tiene otro nombre: código 422.
Tipo del archivo	PDF. Se recomienda indicar `application/pdf` en la parte del archivo; la API reconoce el PDF por su contenido, no por el nombre ni por el tipo declarado.

4.1 Ejemplos por lenguaje

Python (requests)

```
import os
import requests

URL = "http://172.10.30.17:18474/pasaporte"
API_KEY = os.environ["OCR_PASAPORTE_API_KEY"]

def extraer_pasaporte(ruta_pdf: str) -> dict:
    with open(ruta_pdf, "rb") as f:
        resp = requests.post(
            URL,
            headers={"X-API-Key": API_KEY},
            files={"file": (os.path.basename(ruta_pdf), f, "application/pdf")},
            timeout=120,
        )
    return resp.json()          # siempre trae codigo, estado y mensaje

r = extraer_pasaporte("expediente.pdf")
if r["codigo"] in (200, 210, 211, 212):
    d = r["datos"]
    print(d.get("numero_pasaporte"), d.get("nombre_completo"))   # usar .get(): las llaves pueden faltar
```

JavaScript / Node.js 18+ (fetch nativo)

```
import { readFile } from "node:fs/promises";

const URL = "http://172.10.30.17:18474/pasaporte";

export async function extraerPasaporte(rutaPdf) {
  const buffer = await readFile(rutaPdf);
  const form = new FormData();
  form.append("file", new Blob([buffer], { type: "application/pdf" }), "expediente.pdf");

  const resp = await fetch(URL, {
    method: "POST",
    headers: { "X-API-Key": process.env.OCR_PASAPORTE_API_KEY },
    body: form,
    signal: AbortSignal.timeout(120_000),
  });
  return await resp.json();   // { codigo, estado, mensaje, datos?, sello?, mrz?, ... }
}
```

C# (.NET 6+)

```
using System.Net.Http.Headers;
using System.Text.Json;

var http = new HttpClient { Timeout = TimeSpan.FromSeconds(120) };
http.DefaultRequestHeaders.Add("X-API-Key",
    Environment.GetEnvironmentVariable("OCR_PASAPORTE_API_KEY"));

using var form = new MultipartFormDataContent();
var archivo = new ByteArrayContent(await File.ReadAllBytesAsync("expediente.pdf"));
archivo.Headers.ContentType = new MediaTypeHeaderValue("application/pdf");
form.Add(archivo, "file", "expediente.pdf");

var resp = await http.PostAsync("http://172.10.30.17:18474/pasaporte", form);
using var json = JsonDocument.Parse(await resp.Content.ReadAsStringAsync());
int codigo = json.RootElement.GetProperty("codigo").GetInt32();
if (codigo is 200 or 210 or 211 or 212)
{
    var datos = json.RootElement.GetProperty("datos");
    if (datos.TryGetProperty("numero_pasaporte", out var num)) Console.WriteLine(num.GetString());
}
```

PHP (cURL)

```
<?php
$ch = curl_init("http://172.10.30.17:18474/pasaporte");
curl_setopt_array($ch, [
    CURLOPT_POST           => true,
    CURLOPT_HTTPHEADER     => ["X-API-Key: " . getenv("OCR_PASAPORTE_API_KEY")],
    CURLOPT_POSTFIELDS     => ["file" => new CURLFile("expediente.pdf", "application/pdf",
        "expediente.pdf")],
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_TIMEOUT        => 120,
]);
$body = json_decode(curl_exec($ch), true);
curl_close($ch);

if (in_array($body["codigo"], [200, 210, 211, 212])) {
    $numero = $body["datos"]["numero_pasaporte"] ?? null;   // la llave puede no venir
}
```

5. Estructura de la respuesta

Todas las respuestas, exitosas o de error, comparten el mismo sobre. Los bloques de datos solo aparecen cuando aplican. **Regla general: si un dato no se pudo leer, la llave no aparece** (no viene con `null` ni vacía). Accede a los campos de forma segura.

Sobre de la respuesta

```
{
  "codigo": 200,                 // número: qué pasó (sección 6). Úsalo para decidir.
  "estado": "PROCESADO",         // texto fijo equivalente al código
  "mensaje": "...",              // texto para mostrar a una persona; puede cambiar
  "datos": { ... },              // datos de la página de datos      (solo códigos 200-212)
  "sello": { ... },              // sello de cotejo; {} si no hay    (solo códigos 200-212)
  "mrz": { ... },                // lectura y validación de la MRZ   (solo códigos 200-212)
  "validacion": { ... },         // comprobaciones cruzadas          (solo códigos 200-212)
  "avisos": [ ... ],             // puntos a revisar por una persona (solo códigos 200-212)
  "clasificacion": { ... },      // qué documento se detectó         (códigos 200-212, 221 y 225)
  "confianza": { ... },          // nivel de confianza por campo     (solo códigos 200-212)
  "meta": { ... }                // tiempos, páginas, motor          (solo códigos 200-212)
}
```

Las nueve primeras llaves son el contrato homologado con la API de pasaportes de referencia; `confianza` y `meta` son extensiones informativas de este servicio.

5.1 Bloque datos

Campo	Tipo	Ejemplo	Descripción
`tipo_documento`	texto	`P`	Tipo del documento según la página de datos (casi siempre P).
`pais_emisor`	texto	`MEX`	Código ICAO de 3 letras del país que expidió el pasaporte.
`numero_pasaporte`	texto	`G12345678`	Como está impreso, sin espacios.
`apellido_paterno`	texto	`TORRES`	Primera palabra de los apellidos impresos.
`apellido_materno`	texto	`VEGA`	Resto de los apellidos. **No aparece** si el pasaporte trae un solo apellido.
`nombres`	texto	`ANA LUCIA`	Conserva acentos y Ñ como están impresos.
`nacionalidad`	texto	`MEX`	Código de 3 letras de la nacionalidad.
`fecha_nacimiento`	fecha	`1992-07-08`	Formato AAAA-MM-DD.
`lugar_nacimiento`	texto	`CIUDAD DE MEXICO`	Como está impreso.
`sexo`	texto	`F`	**Tal como está impreso**, sin traducir (F, M, H...). Ver pregunta frecuente sobre la M.
`curp`	texto	`TOVA920708MDFRGN02`	18 caracteres. Solo pasaportes mexicanos.
`fecha_expedicion`	fecha	`2020-02-14`	AAAA-MM-DD.
`fecha_vencimiento`	fecha	`2030-02-13`	AAAA-MM-DD.
`autoridad_expedidora`	texto	`SRE`	Autoridad u oficina que expidió el pasaporte.
`nombre_completo`	texto	`ANA LUCIA TORRES VEGA`	Extensión: nombres seguidos de apellidos.
`pais_emisor_nombre`	texto	`MEXICO`	Extensión: nombre del país emisor.
`nacionalidad_impresa`	texto	`MEXICANA`	Extensión: nacionalidad tal como está impresa.
`vigente`	booleano	`true`	Extensión: true si el vencimiento es hoy o posterior.

5.2 Bloque sello

El sello «COTEJADO CONTRA ORIGINAL» que el banco pone sobre la copia. Es **opcional**: si el PDF no lo trae, el bloque es `{}` y no cuenta como falta.

Campo	Tipo	Descripción
`fecha`	fecha	Fecha del sello, AAAA-MM-DD. Si lo leído no es una fecha real se omite y se agrega un aviso.
`reviso`	texto	Nombre de quien revisó, tal como se lee. Una firma sin nombre cuenta como ausente.
`clave_revisor`	texto	Una letra y de 5 a 7 dígitos (S318204). Si no tiene esa forma se omite y se agrega un aviso.

5.3 Bloque mrz

La MRZ son las dos líneas de 44 caracteres al pie de la página de datos. Sus números y fechas traen dígitos de control, por eso es la fuente más confiable. Si no se pudo leer, el bloque es `{}`.

Campo	Tipo	Descripción
`lineas`	lista	Las 2 líneas tal como quedaron después de corregirlas (44 caracteres cada una).
`tipo_documento / pais_emisor`	texto	Tipo (P) y país emisor según la MRZ.
`numero_pasaporte`	texto	Número del pasaporte según la MRZ.
`nacionalidad`	texto	Código de 3 letras.
`numero_personal`	texto	Campo opcional de la MRZ. En pasaportes mexicanos trae el inicio de la CURP.
`fecha_nacimiento / fecha_vencimiento`	fecha	AAAA-MM-DD.
`sexo`	texto	**M, F o X** (convención internacional: M = male, F = female).
`apellido_paterno / apellido_materno / nombres`	texto	Según la MRZ, sin acentos (la MRZ no los admite).
`nombre_abreviado`	booleano	true si el nombre llenó toda la línea y viene recortado.
`digitos_control`	objeto	numero_pasaporte, fecha_nacimiento, fecha_vencimiento, numero_personal y compuesto: true si el dígito cuadra.
`integra`	booleano	true solo si los cinco dígitos de control cuadran.
`origen`	texto	texto (leída por el OCR) o entidad (leída por el modelo de visión).

5.4 Bloque validacion

Campo	Tipo	Descripción
`ok`	booleano	true si no hay discrepancias.
`completo`	booleano	true si se pudieron correr todas las comprobaciones.
`coincidencias`	lista de texto	Comprobaciones que cuadraron.
`discrepancias`	lista de texto	Datos que se contradicen. **Si no está vacía, el código es 210.**
`sin_verificar`	lista de texto	Comprobaciones que no se pudieron hacer (dato faltante, MRZ ilegible, sello a medias).

Comprobaciones que realiza la API, con su etiqueta: `MRZ (dígitos de control)` · `numero_pasaporte (MRZ)` · `fecha_nacimiento (MRZ)` · `sexo (MRZ)` · `fecha_vencimiento (MRZ)` · `nacionalidad (MRZ)` · `apellido_paterno (MRZ)` · `apellido_materno (MRZ)` · `nombres (MRZ)` · `curp vs número personal (MRZ)` · `fecha_nacimiento (CURP)` · `sexo (CURP)`. Los nombres se comparan sin acentos y con Ñ = N, porque la MRZ no los lleva.

Mensaje	Cuándo aparece
`<etiqueta>: hoja de datos 'X' vs 'Y'`	discrepancias: la hoja de datos dice X y la MRZ (o la CURP) dice Y.
`MRZ corrupta: fallan los dígitos de control de ...`	discrepancias: la MRZ no cuadra; no se pudo contrastar la hoja contra ella.
`todas las comprobaciones contra la MRZ`	sin_verificar: acompaña a la MRZ corrupta.
`MRZ completa (no se extrajo)`	sin_verificar: no se encontró la MRZ.
`sello.<campo> (ausente)`	sin_verificar: el sello se encontró, pero le falta ese dato.

5.5 avisos, clasificacion, confianza y meta

Bloque	Contenido
`avisos`	Lista de textos con puntos que conviene revisar a mano, aunque no sean contradicciones: fecha o clave del sello ilegibles, una CURP corregida con la MRZ, o un PDF con más de un pasaporte. **Si no está vacía, el código es 211.**
`clasificacion`	categoria: pasaporte · otro · hoja_en_blanco. caracteres_leidos: cantidad de texto detectado. confianza: siempre null (se conserva por homologación).
`confianza`	Nivel por campo: **alta** (dos fuentes coinciden o lo respalda un dígito de control), **media** (una sola fuente), **baja** (las fuentes se contradicen o no se leyó). Úsalo para resaltar campos en pantallas de revisión.
`meta`	tiempo_total_ms, tiempos_ms, modelo_vlm, paginas, pagina_pasaporte, rotado, recorte, mrz_fuente, otros_documentos (p. ej. tarjeta_td1 si el PDF trae la tarjeta del INM), validaciones_extra y advertencias. Informativo; no tomar decisiones con él.

6. Códigos de respuesta

> **Regla de oro**
> Los códigos **200 a 225 viajan con HTTP 200**, porque la petición fue correcta aunque el documento no sirva. Decide siempre con el campo `codigo` del JSON, no con el estado HTTP. Los errores 4xx y 5xx sí viajan con su mismo estado HTTP.

6.1 Documento procesado (trae datos)

Código	estado	Significado	Qué hacer
200	PROCESADO	Todo cuadró: hoja de datos, MRZ y CURP coinciden.	**Usar los datos directo.**
211	PROCESADO_CON_AVISOS	Sin contradicciones, pero hay algo para revisar (ver avisos). Por ejemplo, la CURP se corrigió con la MRZ o la fecha del sello no se pudo leer.	Usar los datos; revisión rápida de avisos.
212	PROCESADO_INCOMPLETO	Sin contradicciones, pero alguna comprobación no se pudo hacer (ver sin_verificar): falta un dato o el sello viene a medias.	Usar los datos; revisar los campos de sin_verificar.
210	PROCESADO_CON_DISCREPANCIAS	Hay datos que se contradicen (ver discrepancias). Al menos un dato puede estar mal.	**No usar sin revisión humana** de los campos indicados.

6.2 Documento evaluado pero no extraído (sin datos)

Código	estado	Significado	Qué hacer
225	NO_ES_PASAPORTE	El archivo no trae la página de datos de un pasaporte (tarjeta INM, INE, otro).	Pedir el documento correcto.
221	HOJA_EN_BLANCO	No se leyó texto en el documento.	Reescanear con mejor calidad.
222	ARCHIVO_CORRUPTO	El archivo llegó vacío, está dañado, incompleto o no es un PDF.	Reenviar un archivo válido.
223	PDF_PROTEGIDO	El PDF tiene contraseña.	Quitar la contraseña y reenviar.
224	LIMITE_PAGINAS	El PDF tiene más de 30 páginas.	Enviar solo las páginas del pasaporte.

> **225, no 220**
> El código para «no es pasaporte» es **225**. El 220 no se usa en esta API: en la API de credenciales significa «no es INE». Así un mismo cliente distingue los dos casos.

6.3 Errores (sin datos, con su estado HTTP)

Código / HTTP	estado	Causa	Qué hacer	¿Reintentar?
400	ARCHIVO_NO_SOPORTADO	Se envió una imagen u otro tipo: solo se aceptan PDF.	Enviar el PDF.	No
401	NO_AUTORIZADO	Falta el header X-API-Key o la clave es incorrecta.	Revisar configuración.	No
404	NO_ENCONTRADO	Ruta inexistente.	Revisar la URL.	No
405	METODO_NO_PERMITIDO	Método HTTP incorrecto (p. ej. GET a /pasaporte).	Usar POST.	No
413	ARCHIVO_DEMASIADO_GRANDE	Archivo mayor a 25 MB.	Comprimir o enviar solo las páginas del pasaporte.	No
422	PETICION_INVALIDA	Falta el campo file o el multipart está mal formado. La lista errores dice qué falló.	Corregir la petición.	No
500	ERROR_INTERNO	Error no controlado de la API.	Reportar con hora y archivo.	Una vez
429 / 502 / 503	CUOTA_EXCEDIDA / ERROR_DOCUMENT_AI / SIN_CREDENCIALES	Reservados por homologación con la API de referencia; este servicio no los emite.	-	Sí, con espera

Sin respuesta
Si el servidor está caído o no hay red, no llegará ningún JSON: el cliente verá un error de conexión o de timeout. Trátalo como un 500 (reintentar con espera).

7. Integración recomendada

7.1 Cómo decidir

Lógica de decisión (Python)

```
def decidir(body: dict) -> str:
    c = body["codigo"]
    if c == 200:
        return "ACEPTAR"                    # datos listos
    if c in (211, 212):
        return "ACEPTAR_CON_REVISION"       # revisar body["avisos"] /
            body["validacion"]["sin_verificar"]
    if c == 210:
        return "REVISION_OBLIGATORIA"       # revisar body["validacion"]["discrepancias"]
    if c in (221, 225):
        return "DOCUMENTO_INCORRECTO"       # pedir otro documento o reescaneo
    if c in (222, 223, 224, 400, 413, 422):
        return "ARCHIVO_INVALIDO"           # corregir el archivo o la petición; no reintentar igual
    if c == 401:
        return "ERROR_CONFIGURACION"        # API key
    return "REINTENTAR"                     # 500 o cualquier código desconocido
```

7.2 Reintentos

- **Reintentar solo 500 y errores de conexión o timeout** (y 502/503 si algún día aparecen), con espera creciente: 2 s, 4 s y 8 s (máximo 3 intentos).
- **No reintentar** 4xx ni los códigos 210 a 225: el resultado no cambiará enviando el mismo archivo.

Reintentos con espera (Python)

```
import time, requests

def extraer_con_reintentos(ruta, intentos=3):
    for i in range(intentos):
        try:
            body = extraer_pasaporte(ruta)               # función de la sección 4.1
            if body["codigo"] not in (500, 502, 503):
                return body
        except (requests.ConnectionError, requests.Timeout):
            pass
        time.sleep(2 ** (i + 1))                         # 2 s, 4 s, 8 s
    raise RuntimeError("OCR de pasaportes no disponible después de varios intentos")
```

7.3 Concurrencia

La API procesa **hasta 2 expedientes a la vez** y comparte la GPU con la API de credenciales INE. Si llegan más, esperan su turno sin error y se atienden en cuanto se libera un lugar.

Peticiones enviadas a la vez	Tiempo aproximado de la última respuesta
1 a 2	15 a 25 segundos
3 a 4	30 a 50 segundos
5 a 6	45 a 75 segundos

- **Recomendado:** en cargas masivas, que el cliente envíe como máximo 2 en paralelo; enviar más no acelera, solo hace esperar.
- Mantén el timeout de 120 segundos: cubre la espera de turno en cargas normales.
- El límite lo ajusta el responsable del servicio (MAX_CONCURRENT_JOBS).

7.4 Buenas prácticas

- Guarda el JSON completo junto al expediente: permite auditar qué devolvió la API y por qué.
- Muestra en pantallas de revisión los textos de `discrepancias`, `avisos` y `sin_verificar`, y resalta los campos con confianza `media` o `baja`.
- Trata cualquier código desconocido como error y regístralo: permite agregar códigos nuevos sin romper la integración.
- No registres en logs el contenido de datos (nombres, número de pasaporte, CURP): son datos personales.

8. Ejemplos de respuesta

Ejemplos generados por el código real de la API con **datos ficticios** (`scripts/generar_ejemplos.py`).

8.1 Código 200: procesado

200 PROCESADO (respuesta completa, meta resumido)


```
{
  "codigo": 200,
  "estado": "PROCESADO",
  "mensaje": "El documento se procesó y todas las comprobaciones cuadraron.",
  "datos": {
    "tipo_documento": "P",
    "pais_emisor": "MEX",
    "numero_pasaporte": "G12345678",
    "apellido_paterno": "TORRES",
    "apellido_materno": "VEGA",
    "nombres": "ANA LUCIA",
    "nacionalidad": "MEX",
    "fecha_nacimiento": "1992-07-08",
    "lugar_nacimiento": "CIUDAD DE MEXICO",
    "sexo": "F",
    "curp": "TOVA920708MDFRGN02",
    "fecha_expedicion": "2020-02-14",
    "fecha_vencimiento": "2030-02-13",
    "autoridad_expedidora": "SRE",
    "nombre_completo": "ANA LUCIA TORRES VEGA",
    "pais_emisor_nombre": "MEXICO",
    "nacionalidad_impresa": "MEXICANA",
    "vigente": true
  },
  "sello": {
    "fecha": "2026-01-15",
    "reviso": "Jorge Luis Medina Ortiz",
    "clave_revisor": "S318204"
  },
  "mrz": {
    "lineas": [
      "P<MEXTORRES<VEGA<<ANA<LUCIA<<<<<<<<<<<<<<<<",
      "G123456786MEX9207086F3002131TOVA920708MDFR78"
    ],
    "tipo_documento": "P",
    "pais_emisor": "MEX",
    "numero_pasaporte": "G12345678",
    "nacionalidad": "MEX",
    "numero_personal": "TOVA920708MDFR",
    "fecha_nacimiento": "1992-07-08",
    "sexo": "F",
    "fecha_vencimiento": "2030-02-13",
    "apellido_paterno": "TORRES",
    "apellido_materno": "VEGA",
    "nombres": "ANA LUCIA",
    "nombre_abreviado": false,
    "digitos_control": {
      "numero_pasaporte": true,
      "fecha_nacimiento": true,
      "fecha_vencimiento": true,
      "numero_personal": true,
      "compuesto": true
    },
    "integra": true,
    "origen": "texto"
  },
```

```
  "validacion": {
    "ok": true,
    "completo": true,
    "coincidencias": [
      "MRZ (dígitos de control)",
      "numero_pasaporte (MRZ)",
      "fecha_nacimiento (MRZ)",
      "sexo (MRZ)",
      "fecha_vencimiento (MRZ)",
      "nacionalidad (MRZ)",
      "apellido_paterno (MRZ)",
      "apellido_materno (MRZ)",
      "nombres (MRZ)",
      "curp vs número personal (MRZ)",
      "fecha_nacimiento (CURP)",
      "sexo (CURP)"
    ],
    "discrepancias": [],
    "sin_verificar": []
  },
  "avisos": [],
  "clasificacion": {
    "categoria": "pasaporte",
    "confianza": null,
    "caracteres_leidos": 412
  },
  "confianza": {
    "pais_emisor": "alta",
    "numero_pasaporte": "alta",
    "apellidos": "alta",
    "nombres": "alta",
    "fecha_nacimiento": "alta",
    "fecha_vencimiento": "alta",
    "sexo": "alta",
    "nacionalidad": "alta",
    "fecha_expedicion": "alta",
    "lugar_nacimiento": "media",
    "autoridad": "media",
    "curp": "alta"
  },
  "meta": "{ ... tiempos, páginas, motor ... }"
}
```


8.2 Código 210: con discrepancias

La hoja de datos se leyó con un dígito distinto en el número de pasaporte. La MRZ, respaldada por su dígito de control, trae el número correcto: requiere revisión.

210 PROCESADO_CON_DISCREPANCIAS (extracto)

```
{
  "codigo": 210,
  "estado": "PROCESADO_CON_DISCREPANCIAS",
  "mensaje": "El documento se procesó, pero hay datos que se contradicen entre sí. Revisa
      'validacion.discrepancias' antes de dar los datos por buenos.",
  "datos": "{ ... igual que en 200 ... }",
  "validacion": {
    "ok": false,
    "completo": true,
    "coincidencias": [
      "MRZ (dígitos de control)",
      "fecha_nacimiento (MRZ)",
      "sexo (MRZ)",
      "fecha_vencimiento (MRZ)",
      "nacionalidad (MRZ)",
      "apellido_paterno (MRZ)",
      "apellido_materno (MRZ)",
      "nombres (MRZ)",
      "curp vs número personal (MRZ)",
      "fecha_nacimiento (CURP)",
      "sexo (CURP)"
    ],
    "discrepancias": [
      "numero_pasaporte (MRZ): hoja de datos 'G12345673' vs 'G12345678'"
    ],
    "sin_verificar": []
  },
  "avisos": []
}
```

8.3 Código 211: con avisos

La CURP se leyó con el dígito verificador equivocado; se corrigió con la fecha y el sexo de la MRZ y se avisa.

211 PROCESADO_CON_AVISOS (extracto)

```
{
  "codigo": 211,
  "estado": "PROCESADO_CON_AVISOS",
  "mensaje": "El documento se procesó y nada se contradijo, pero hay algo que requiere revisión humana.
      Revisa 'avisos'.",
  "datos": "{ ... igual que en 200 ... }",
  "validacion": {
    "ok": true,
    "completo": true,
    "coincidencias": [
      "MRZ (dígitos de control)",
      "numero_pasaporte (MRZ)",
      "fecha_nacimiento (MRZ)",
      "sexo (MRZ)",
```

```
      "fecha_vencimiento (MRZ)",
      "nacionalidad (MRZ)",
      "apellido_paterno (MRZ)",
      "apellido_materno (MRZ)",
      "nombres (MRZ)",
      "curp vs número personal (MRZ)",
      "fecha_nacimiento (CURP)",
      "sexo (CURP)"
    ],
    "discrepancias": [],
    "sin_verificar": []
  },
  "avisos": [
    "datos.curp corregida con la MRZ y el dígito verificador: se leyó 'TOVA920708MDFRGN07' y quedó
        'TOVA920708MDFRGN02'"
  ]
}
```

8.4 Código 212: incompleto

El sello trae fecha, pero en REVISÓ solo hay una firma y no viene la clave del revisor.

212 PROCESADO_INCOMPLETO (extracto)

```
{
  "codigo": 212,
  "estado": "PROCESADO_INCOMPLETO",
  "mensaje": "El documento se procesó y nada se contradijo, pero quedaron comprobaciones sin poder
      realizarse. Revisa 'validacion.sin_verificar'.",
  "sello": {
    "fecha": "2026-01-15"
  },
  "validacion": {
    "ok": true,
    "completo": false,
    "coincidencias": [
      "MRZ (dígitos de control)",
      "numero_pasaporte (MRZ)",
      "fecha_nacimiento (MRZ)",
      "sexo (MRZ)",
      "fecha_vencimiento (MRZ)",
      "nacionalidad (MRZ)",
      "apellido_paterno (MRZ)",
      "apellido_materno (MRZ)",
      "nombres (MRZ)",
      "curp vs número personal (MRZ)",
      "fecha_nacimiento (CURP)",
      "sexo (CURP)"
    ],
    "discrepancias": [],
    "sin_verificar": [
      "sello.reviso (ausente)",
      "sello.clave_revisor (ausente)"
    ]
  }
}
```

8.5 Código 225 y 221: sin datos

225 NO_ES_PASAPORTE

```
{
  "codigo": 225,
  "estado": "NO_ES_PASAPORTE",
  "mensaje": "El documento no es un pasaporte. No se extrajo ningun dato.",
  "clasificacion": {
    "categoria": "otro",
    "confianza": null,
    "caracteres_leidos": 640
  }
}
```

221 HOJA_EN_BLANCO

```
{
  "codigo": 221,
  "estado": "HOJA_EN_BLANCO",
  "mensaje": "El documento viene en blanco o ilegible: no se leyó ningún texto.",
  "clasificacion": {
    "categoria": "hoja_en_blanco",
    "confianza": null,
    "caracteres_leidos": 0
  }
}
```

8.6 Errores

Errores (una línea por código)

```
// HTTP 401
{"codigo": 401, "estado": "NO_AUTORIZADO", "mensaje": "Falta o no coincide el header X-API-Key."}

// HTTP 400
{"codigo": 400, "estado": "ARCHIVO_NO_SOPORTADO", "mensaje": "El tipo de archivo enviado no se acepta."}

// HTTP 413
{"codigo": 413, "estado": "ARCHIVO_DEMASIADO_GRANDE", "mensaje": "El archivo excede el tamaño máximo.
    Máximo 25 MB."}

// HTTP 422
{"codigo": 422, "estado": "PETICION_INVALIDA", "mensaje": "La petición no es válida. Se espera
    multipart/form-data con un campo 'file'.", "errores": ["file: Field required"]}
// HTTP 200
{"codigo": 222, "estado": "ARCHIVO_CORRUPTO", "mensaje": "El archivo no se pudo abrir: está dañado,
    incompleto, o no es del tipo que dice ser."}
```

9. Health check

`GET /health` no requiere API key. Úsalo para monitoreo o para verificar la conexión antes de integrar.

Petición y respuesta

```
curl http://172.10.30.17:18474/health

{"status": "ok", "vlm": true, "modelo_vlm": "qwen3-vl:8b-instruct", "backend_vlm": "ollama",
    "formato_respuesta": "homologado", "max_concurrent_jobs": 2}
```

`status` vale `degradado` si el modelo de visión no responde: el servicio está arriba pero la extracción saldría incompleta. La prueba completa es enviar un documento conocido.

10. Seguridad

- **API key:** trátala como contraseña. Guárdala en variables de entorno o un gestor de secretos; no la subas a repositorios ni la compartas por correo o chat. Si se filtra, el responsable del servicio la cambia y la anterior deja de funcionar al instante.
- **Red:** el servicio usa HTTP sin cifrado y está pensado para la red interna. No lo expongas a internet; si se necesita desde fuera, debe ir detrás de un proxy con HTTPS.
- **Datos personales:** todo se procesa en el servidor de la empresa con modelos locales; los documentos no se envían a ningún servicio externo. La API no guarda los archivos ni las respuestas.
- **Logs:** el servicio solo registra código, páginas y tiempo de cada petición. No registres datos de clientes en logs de aplicaciones cliente.

11. Precisión y limitaciones

Medido sobre un lote real de 503 expedientes, de los cuales 481 traen pasaporte, de más de 40 países. Son indicadores de consistencia entre fuentes independientes; la medición campo por campo contra una captura manual está pendiente.

Indicador	Resultado	Cómo se midió
Expedientes resueltos sin revisión	**94.6 %**	Pasaportes en los que todos los campos críticos quedaron confirmados.
MRZ íntegra	**92 %**	Todos los dígitos de control de la MRZ cuadran.
Número, nacimiento y vencimiento confirmados	97 a 98 %	Respaldados por su dígito de control y coincidentes con la hoja.
Apellidos y nombres confirmados	96 a 97 %	La hoja de datos y la MRZ dicen lo mismo.
CURP válida (pasaportes mexicanos)	87 %	Estructura, dígito verificador, fecha y sexo coinciden.
No pasaporte detectado	21 de 21	Expedientes con solo tarjeta INM u otros documentos: código 225.

- **Lectura impresa contra MRZ:** datos entrega lo que dice la hoja de datos. Si el modelo la leyó distinto de la MRZ, la respuesta es 210 y la discrepancia trae ambos valores; en esos casos casi siempre la MRZ es la correcta.
- **Apellidos:** la división en paterno y materno es la regla homologada (primera palabra y resto). En apellidos compuestos (DE LA CRUZ) la división no es natural, aunque la comparación contra la MRZ sí los acepta.
- **Sexo M en pasaportes mexicanos:** es ambiguo (mujer en español, male en el estándar), por eso no se compara.
- Si un PDF trae dos pasaportes, se entrega el primero y se agrega un aviso.

- El modelo de visión no es totalmente determinista: el mismo archivo puede variar en campos sin dígito de control (autoridad, lugar de nacimiento).
- Copias muy tenues, sellos sobre la MRZ o páginas recortadas bajan la precisión; la API lo refleja con 210, 211 o 212.

12. Preguntas frecuentes

Recibo HTTP 200 pero no hay datos.

Revisa `codigo`: probablemente es 221, 222, 223, 224 o 225. Esos códigos viajan con HTTP 200.

¿Por qué falta una llave en datos?

Porque ese dato no se pudo leer o no existe en el pasaporte (p. ej. apellido_materno con un solo apellido, o curp en pasaportes extranjeros). Usa acceso seguro (.get, ??, TryGetProperty).

¿Por qué datos.sexo dice M y mrz.sexo dice F?

datos.sexo es lo impreso: en pasaportes mexicanos «M» puede significar mujer. La MRZ usa la convención internacional (F = female). Es la misma persona.

El sello viene vacío ({}).

El PDF no trae el sello de cotejo o no se pudo localizar. No es una falta: no cambia el código.

Recibo 212 por el sello.

El sello se encontró pero le falta la fecha, el nombre de quien revisó o la clave. Los datos del pasaporte son válidos.

El expediente trae pasaporte y tarjeta INM.

No pasa nada: la API toma la página del pasaporte y lo reporta en meta.otros_documentos.

El expediente solo trae la tarjeta INM.

Responde 225: no hay pasaporte.

Me responde 401.

Falta el header X-API-Key, tiene otro nombre o la clave es incorrecta. Las mayúsculas del nombre del header no importan; el valor sí.

Me responde 422.

El archivo no viaja en el campo file o la petición no es multipart/form-data. La lista errores indica qué faltó.

¿Cuál uso, /pasaporte o /extract?

/pasaporte. /extract es un alias para pruebas.

13. Prompt para implementar con IA

Copia este prompt en un asistente de IA (Claude, ChatGPT, Copilot...) e indica el lenguaje o framework. Contiene el contrato completo para generar un cliente correcto. También está en el archivo `docs/PROMPT_IA.md` del proyecto, más fácil de copiar.

Prompt

```
Actúa como desarrollador senior. Implementa en [LENGUAJE / FRAMEWORK] un cliente para la API
"OCR de Pasaportes" siguiendo EXACTAMENTE este contrato. Entrega: módulo cliente, modelos tipados
de la respuesta, manejo de errores y reintentos, y un ejemplo de uso.

== QUÉ HACE LA API ==
Recibe un expediente escaneado en PDF y devuelve en JSON los datos de la página de datos del
pasaporte (de cualquier país), validados contra su zona MRZ. Ignora cualquier otro documento del
archivo (tarjeta de residente del INM, credencial INE, solicitudes, etc.). Si no hay pasaporte
responde el código 225.

== PETICIÓN ==
POST http://172.10.30.17:18474/pasaporte
Header obligatorio: X-API-Key: <clave>   (leerla de una variable de entorno OCR_PASAPORTE_API_KEY;
                                           nunca escribirla en el código)
Body: multipart/form-data con UN archivo PDF en el campo "file" (content-type application/pdf).
Solo se aceptan PDF. Tamaño máximo 25 MB. Máximo 30 páginas por archivo; la página de datos del
pasaporte debe estar en las primeras 4 y debe incluir las 2 líneas MRZ del pie.
Timeout del cliente: 120 s. La API tarda de 10 a 25 s por documento y procesa hasta 2 a la vez;
las demás esperan turno. En cargas masivas, enviar como máximo 2 peticiones en paralelo.
POST /extract es un alias para pruebas que responde igual. GET /health (sin API key) devuelve
{"status":"ok",...} para monitoreo.

== RESPUESTA: SIEMPRE ESTE SOBRE JSON ==
{
  "codigo": int,          // decide con ESTE campo, no con el estado HTTP
  "estado": str,          // nombre fijo del código
  "mensaje": str,         // texto para humanos
  "datos": {...},         // solo códigos 200, 210, 211, 212
  "sello": {...},         // solo 200-212; puede ser {} si el PDF no trae sello de cotejo
  "mrz": {...},           // solo 200-212; {} si no se pudo leer la MRZ
  "validacion": {"ok": bool, "completo": bool, "coincidencias": [str],
                 "discrepancias": [str], "sin_verificar": [str]},   // solo 200-212
  "avisos": [str],        // solo 200-212
  "clasificacion": {"categoria": "pasaporte"|"otro"|"hoja_en_blanco",
                    "confianza": null, "caracteres_leidos": int},   // 200-212, 221 y 225
  "confianza": {"<campo>": "alta"|"media"|"baja"},   // solo 200-212 (extensión)
  "meta": {...}           // solo 200-212 (extensión, informativo)
}
REGLA: si un dato no se pudo leer, la LLAVE NO APARECE (no viene null ni ""). Todos los campos de
"datos", "sello", "mrz", "confianza" y "meta" deben modelarse como OPCIONALES.

== CAMPOS DE "datos" ==
tipo_documento (str, normalmente "P") · pais_emisor (str, código ICAO de 3 letras: "MEX")
numero_pasaporte (str) · apellido_paterno (str: primera palabra de los apellidos)
apellido_materno (str: el resto de los apellidos; NO aparece si el pasaporte trae un solo apellido)
nombres (str) · nacionalidad (str, código de 3 letras: "MEX")
fecha_nacimiento ("AAAA-MM-DD") · lugar_nacimiento (str)
sexo (str, TAL COMO ESTÁ IMPRESO: "F", "M", "H"...; en pasaportes mexicanos "M" puede ser mujer)
curp (str, 18 chars, solo pasaportes mexicanos) · fecha_expedicion ("AAAA-MM-DD")
fecha_vencimiento ("AAAA-MM-DD") · autoridad_expedidora (str)
Extensiones: nombre_completo (str) · pais_emisor_nombre (str) · nacionalidad_impresa (str) · vigente
    (bool)

== CAMPOS DE "sello" (sello "COTEJADO CONTRA ORIGINAL" del banco) ==
fecha ("AAAA-MM-DD") · reviso (str, nombre de quien revisó) · clave_revisor (str: una letra + 5 a 7
    dígitos)
```

```
== CAMPOS DE "mrz" ==
lineas ([str] x2, 44 caracteres cada una), tipo_documento, pais_emisor, numero_pasaporte,
nacionalidad, numero_personal (opcional), fecha_nacimiento, sexo ("M" | "F" | "X" -- convención
internacional), fecha_vencimiento, apellido_paterno, apellido_materno, nombres,
nombre_abreviado (bool), digitos_control {numero_pasaporte, fecha_nacimiento, fecha_vencimiento,
numero_personal, compuesto: bool}, integra (bool: true solo si los cinco cuadran),
origen ("texto" | "entidad").

== CÓDIGOS Y ACCIÓN ==
Viajan con HTTP 200 (la petición fue válida):
 200 PROCESADO                    -> ACEPTAR: usar datos directo.
 211 PROCESADO_CON_AVISOS         -> ACEPTAR_CON_REVISION: mostrar "avisos".
 212 PROCESADO_INCOMPLETO         -> ACEPTAR_CON_REVISION: mostrar validacion.sin_verificar.
 210 PROCESADO_CON_DISCREPANCIAS  -> REVISION_OBLIGATORIA: mostrar validacion.discrepancias;
                                     no usar los datos sin revisión humana.
 225 NO_ES_PASAPORTE              -> DOCUMENTO_INCORRECTO (sin datos).
 221 HOJA_EN_BLANCO               -> DOCUMENTO_INCORRECTO (reescanear).
 222 ARCHIVO_CORRUPTO             -> ARCHIVO_INVALIDO.
 223 PDF_PROTEGIDO                -> ARCHIVO_INVALIDO (quitar contraseña).
 224 LIMITE_PAGINAS               -> ARCHIVO_INVALIDO (más de 30 páginas).
Viajan con su mismo estado HTTP:
 400 ARCHIVO_NO_SOPORTADO, 413 ARCHIVO_DEMASIADO_GRANDE, 422 PETICION_INVALIDA -> ARCHIVO_INVALIDO
 401 NO_AUTORIZADO, 404 NO_ENCONTRADO, 405 METODO_NO_PERMITIDO               -> ERROR_CONFIGURACION
 500 ERROR_INTERNO                                                            -> REINTENTAR
 429, 502, 503: reservados por homologación (hoy no se emiten)                -> REINTENTAR
Cualquier código desconocido -> tratar como REINTENTAR y registrarlo.
OJO: 220 NO se usa en esta API (es "no es INE" en la API de credenciales).
== REINTENTOS ==
Reintentar SOLO 500, 502, 503, errores de conexión y timeouts: máximo 3 intentos con espera
de 2 s, 4 s y 8 s. Nunca reintentar 4xx ni 210-225.

== REQUISITOS DE IMPLEMENTACIÓN ==
1. Función principal: extraer(archivo) -> resultado tipado con: codigo, estado, mensaje,
   accion (enum de las acciones de arriba), datos (opcional), sello (opcional), avisos,
   discrepancias, sin_verificar y el JSON crudo completo.
2. Leer SIEMPRE el JSON aunque el HTTP sea 4xx/5xx (también trae el sobre).
3. Si la respuesta no es JSON (caída del servidor, proxy), devolver accion=REINTENTAR.
4. No registrar en logs nombres, número de pasaporte, CURP ni el contenido de "datos" (datos
    personales).
5. API key, URL y timeout configurables por variables de entorno.
6. Incluir pruebas unitarias con respuestas simuladas para 200, 210, 212, 225, 401 y 500.

== EJEMPLO DE RESPUESTA 200 (datos ficticios, extracto) ==
{"codigo":200,"estado":"PROCESADO","mensaje":"El documento se procesó y todas las comprobaciones
    cuadraron.",
 "datos":{"tipo_documento":"P","pais_emisor":"MEX","numero_pasaporte":"G12345678","apellido_paterno":"TO
     RRES",
  "apellido_materno":"VEGA","nombres":"ANA LUCIA","nacionalidad":"MEX","fecha_nacimiento":"1992-07-08",
  "lugar_nacimiento":"CIUDAD DE
      MEXICO","sexo":"F","curp":"TOVA920708MDFRGN02","fecha_expedicion":"2020-02-14",
  "fecha_vencimiento":"2030-02-13","autoridad_expedidora":"SRE"},
 "sello":{"fecha":"2026-01-15","reviso":"Jorge Luis Medina Ortiz","clave_revisor":"S318204"},
 "validacion":{"ok":true,"completo":true,"coincidencias":["MRZ (dígitos de control)","numero_pasaporte
     (MRZ)"],
  "discrepancias":[],"sin_verificar":[]},
 "avisos":[],"clasificacion":{"categoria":"pasaporte","confianza":null,"caracteres_leidos":412}}
```

14. Glosario

Término	Significado
Página de datos	La página del pasaporte con la foto, los datos del titular y la MRZ.
MRZ	Zona de lectura mecánica: las 2 líneas de 44 caracteres con «<» al pie de la página de datos. Formato ICAO TD3.
ICAO 9303	Estándar internacional de documentos de viaje que define la MRZ y sus códigos de país.
Dígito de control	Número calculado a partir de un dato de la MRZ que permite detectar errores de lectura.
Número personal	Campo opcional de la MRZ; en pasaportes mexicanos trae el inicio de la CURP.
CURP	Clave Única de Registro de Población (18 caracteres), en pasaportes mexicanos.
Sello de cotejo	Sello «COTEJADO CONTRA ORIGINAL» que el banco pone en la copia, con fecha, quién revisó y su clave.
PaddleOCR / Qwen3-VL	Modelos open source que leen el texto y la imagen del pasaporte en el servidor de la empresa.